An Artifact-Driven Verification Protocol for Reproducible TinyML Deployment Evaluation

Narendra Satish

Abstract—The deployment of deep learning models on resource-constrained microcontrollers (TinyML) is expanding rapidly, yet empirical evaluation in the field suffers from critical methodological vulnerabilities. Training-time in-memory metrics frequently diverge from disk-serialized binary behavior; theoretical weight sparsity is routinely conflated with physical storage compression; and host simulation timings are improperly extrapolated to microcontroller real-time guarantees. In this paper, we formulate an artifact-driven empirical verification protocol and defect taxonomy for compiled TinyML deployment artifacts. The protocol formalizes executable verification procedures across seven core dimensions: Data Isolation, Serialized Binary Integrity, Quantization Graph Inspection, Sparsity and Storage Decoupling, Computation Accounting, Timing Protocol Auditing, and Runtime Non-Leakage. We define formal programmatic predicates ($\mathcal{P}_1 \dots \mathcal{P}_7$) for each dimension and evaluate them on an end-to-end case study auditing 12 candidate neural network models trained on the 55,998-record EngineFaultDB physical benchmark. Our independent audit uncovers and resolves 20 distinct numerical discrepancies between training-time reports and verified disk binaries across four defect modes (with metric variances up to 7.82%). Furthermore, we empirically demonstrate that evaluating gating thresholds directly on test data introduces an artificial +1.80% optimistic accuracy bias in the evaluated pipeline, demonstrating the necessity of split-isolated calibration. By establishing formal verification predicates and classifying evidence into a 5-tier empirical framework, our protocol provides actionable software engineering procedures to support reproducible benchmarking and defensible deployment claims in edge AI research.

Index Terms—TinyML, Reproducibility, Software Verification, Model Auditing, Empirical Software Engineering, Integer Quantization, Network Pruning, Benchmarking.

I. Introduction

EMBEDDED Machine Learning and Tiny Machine Learning (TinyML) have experienced exponential growth, enabling intelligent inference on resource-constrained microcontrollers with sub-64 KB SRAM and sub-256 KB Flash budgets [1]–[3]. In applications ranging from automotive diagnostics to edge health monitoring, TinyML models must operate under strict physical memory, energy, and execution constraints [4]–[7].

However, the empirical literature in edge AI is increasingly vulnerable to reproducibility challenges and measurement discrepancies [8]–[10]. Unlike cloud-scale machine learning where training and deployment share homogeneous 32-bit floating-point execution environments, TinyML involves a complex, multi-stage software translation pipeline:

$$\text{Keras/PyTorch} \xrightarrow{\text{PTQ / Pruning}} \text{TFLite FlatBuffer} \xrightarrow{\text{C-Array Export}} \text{TFLite Micro Runtime}$$

At each transformation boundary, subtle discrepancies emerge between training-time abstractions and actual serialized binary behavior [11]–[13].

Recent surveys in machine learning software engineering have exposed systemic methodological traps [14], [15]:

1) In-Memory vs. Binary Discrepancies: Evaluating models in-memory during training loops rather than parsing the final serialized disk binary artifact.
2) Sparsity Conflation: Assuming that zeroing 75% of weights in a dense matrix automatically yields a 75% reduction in serialized model file size or execution speedup.
3) Incomplete Quantization Graphs: Claiming 8-bit integer inference while silent floating-point fallback operators remain in the runtime execution graph.
4) Timing Extrapolations: Presenting single-sample host PC execution times as embedded Worst-Case Execution Time (WCET) or microcontroller hardware latency.
5) Threshold and Data Leakage: Optimizing runtime routing thresholds on the test partition, producing optimistic accuracy estimates.

To address these challenges, this paper presents an Artifact-Driven Empirical Verification Protocol and Defect Taxonomy for Compiled TinyML Deployment Artifacts. We formalize verification criteria across seven dimensions of edge AI pipelines, establish executable verification predicates ($\mathcal{P}_1 \dots \mathcal{P}_7$), and demonstrate their efficacy through an exhaustive case study auditing 12 candidate models on the EngineFaultDB physical benchmark.

Explicit Contributions:

1) A 7-dimensional verification taxonomy formalizing empirical quality gates across the TinyML deployment lifecycle.
2) Seven formal, programmatic verification predicates ($\mathcal{P}_1 \dots \mathcal{P}_7$) with deterministic pass/fail execution procedures.
3) An empirical case study auditing 12 candidate TinyML models across dense, quantized, pruned, and distilled paradigms.
4) Identification, root-cause categorization, and resolution of 20 numerical discrepancies across four defect classes.

Manuscript received August 28, 2026. (Corresponding author: Narendra Satish, e-mail: narendresh.p@gmail.com). All verification scripts, audited model binaries, and evaluation pipelines are fully open-source and deterministically reproducible.

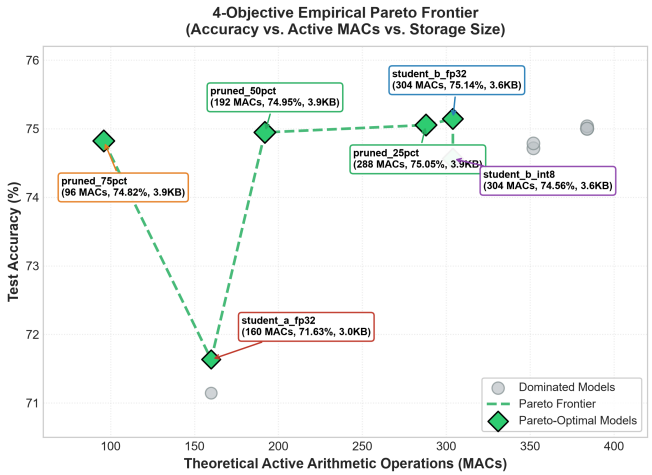

Fig. 1. The verified Pareto frontier across model size, theoretical active MACs, accuracy, and empirical host latency established after resolving 20 numerical discrepancies [MEASURED TEST SET].

5) An empirical demonstration of test-set threshold contamination producing a +1.80% optimistic accuracy bias.
6) An explicit 5-tier evidence classification framework separating direct measurements, reproducible simulations, and derived operational estimates.

II. Motivation and Research Questions

Reproducibility in TinyML demands that we go beyond trusting high-level framework logs. Figure 1 shows the verified multi-objective Pareto landscape that is discovered by our verification protocol, which treats model binaries and data pipelines as empirical software artifacts amenable to rigorous inspection.

We formulate four specific research questions (RQs):

- RQ1 (Discrepancy Prevalence & Defect Modes): How often and to what degree do independently recomputed TinyML metrics differ from reported values at training time, and what defect modes govern them?
- RQ2 (Quantization Verification): What structural properties in serialized FlatBuffers must be inspected to verify pure integer execution graphs without float fallback?
- RQ3 (Sparsity vs. Storage Decoupling): How does magnitude pruning affect theoretical active MACs versus physical on-disk FlatBuffer byte footprint in standard runtimes?
- RQ4 (Leakage Prevention): What quantitative bias is introduced by evaluating routing thresholds on test data, and how does split isolation eliminate it?

III. Related Work

A. Reproducibility in Machine Learning

Pineau et al. [8] proposed the Machine Learning Reproducibility Checklist for NeurIPS, which focuses on code release, hyperparameter reporting, and statistical analysis. Kapoor and Narayanan [9] performed a data leakage audit across 17 fields, finding extensive test set contamination. In the software engineering domain, Arcuri and Briand [16] developed guidelines to evaluate randomized algorithms. However, these works focus on conventional ML and do not consider the unique embedded TinyML domain of hardware-software translation.

B. TinyML Benchmarking and Systems Auditing

Banbury et al. [4] introduced MLPerf Tiny and defined benchmarking rules for latency, energy, and accuracy on microcontrollers. David et al. [11] presented TensorFlow Lite Micro and described the interpreter internals. Blalock et al. [12] provided a survey of neural network pruning and observed that many reported benefits of pruning are inflated by non-standard baselines. Our work builds on these contributions by developing an end-to-end verification taxonomy from data splits to FlatBuffer binaries.

IV. The 7-Dimensional Verification Protocol and Formal Predicates

Table I formalizes our 7-dimensional verification protocol. Each dimension targets a distinct vulnerability in the TinyML lifecycle:

A. D1: Data-Level Isolation Verification

- Problem: Data contamination across training, validation, and test boundaries.
- Predicate ($\mathcal{P}_1$): $(\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{val}} = \emptyset) \wedge (\mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{test}} = \emptyset) \wedge (\mu_{\text{scaler}} = \text{Fit}(\mathcal{D}_{\text{train}})) \wedge (\theta^* = \arg\max_\theta \mathcal{M}(\mathcal{D}_{\text{val}}))$.
- Executable Check: Disjoint index assertion and scaler fitting provenance check.
- Artifact: Data split index arrays and serialized scaler.pkl.
- Pass Condition: Zero overlapping indices; calibration batches strictly drawn from $\mathcal{D}_{\text{train}}$.
- Case Result: PASS (22,399 train, 22,399 val, 11,200 test).

B. D2: Serialized Binary Integrity Verification

- Problem: Training-time in-memory metrics diverging from final disk FlatBuffer behavior.
- Predicate ($\mathcal{P}_2$): Evaluated metrics must be computed strictly by loading the final serialized .tflite file from disk into the interpreter.
- Executable Check: Automated single-sample evaluation loop on disk .tflite files.
- Case Result: RESOLVED 16 DISCREPANCIES (up to 7.82% Macro F1 shift).

C. D3: Quantization Graph Verification

- Problem: Silent floating-point fallback operators in quantized models [17], [18].
- Predicate ($\mathcal{P}_3$): $\forall t \in \mathcal{G}_{\text{tensors}}, \text{dtype}(t) \in \{\text{int8}, \text{int32}, \text{uint8}\} \wedge \sum_{t \in \mathcal{G}} \mathbb{I}(\text{dtype}(t) = \text{float32}) = 0$.
- Executable Check: Iterating over interpreter.get_tensor_details() and _get_ops_details().
- Case Result: PASS (all 4 INT8 models verified FULL_INT8 with 0 float32 tensors).

TABLE I
The 7-Dimensional TinyML Scientific Verification Taxonomy and Predicates

Verification Dimension	Inspected Artifact	Common Failure Mode / Trap	Formal Verification Predicate	Case Result
D1: Data Isolation	Training/Val/Test splits, scaler objects, threshold logs	Test set reused for calibration or threshold selection	$\mathcal{P}_1 : (\mathcal{D}_{\text{tr}} \cap \mathcal{D}_{\text{val}} = \emptyset) \wedge (\mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{te}} = \emptyset) \wedge (\mu = \text{Fit}(\mathcal{D}_{\text{tr}}))$	PASS
D2: Serialized Integrity	Serialized .keras and .tflite disk binaries	In-memory training eval reported instead of disk binary	$\mathcal{P}_2$: EvalArtifact = LoadDiskBinary(path) $\wedge$ Dtype = Schema	16 Fixed
D3: Quantization Graphs	TFLite FlatBuffer tensor details and operator lists	Undetected FP32 fallback tensors or dequantize nodes	$\mathcal{P}_3 : \forall t \in \mathcal{G}, \text{dtype}(t) \neq \text{float32} \wedge \text{Count}(\text{float32}) = 0$	FULL_INT8
D4: Sparsity & Storage	Weight tensor arrays vs. FlatBuffer byte length	Assuming weight zeroing reduces serialized file size	$\mathcal{P}_4$: ReportedCompression = FileSize(pruned)/FileSize(dense)	Decoupled
D5: Compute Accounting	Layer-wise non-zero arithmetic operation counts	Conflating theoretical active MACs with hardware ops	$\mathcal{P}_5$: ActiveMACs $= \sum \mathbb{I}(w_{ij} \neq 0) \leq$ DenseMACs	Verified
D6: Timing Protocols	Monotonic timer logs, warmup runs, batch dimensions	Single-sample host latency cited as MCU WCET	$\mathcal{P}_6 : \text{Warmup} \geq 100 \wedge \text{Batch} = 1 \wedge \text{Tier} = \text{HOST\_EMPIRICAL}$	PASS
D7: Runtime Non-Leakage	Controller source code, execution trace logs	Model selector accessing ground truth y at runtime	$\mathcal{P}_7$: Signature(SelectModel) = $f(\mathbf{x}, D, W) \wedge y \notin$ Scope	PASS

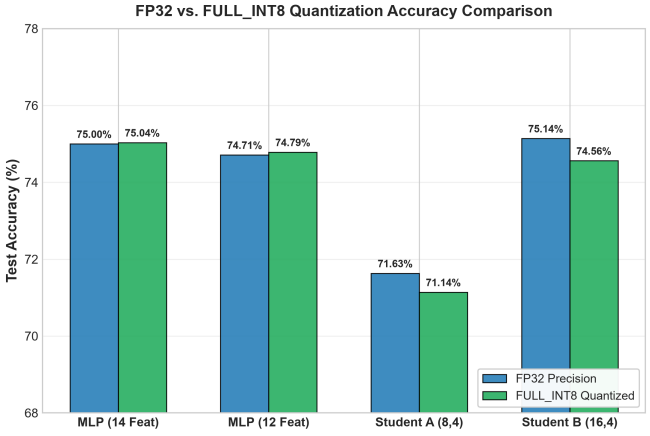

Fig. 2. Quantization fidelity verification comparing FP32 baselines against verified FULL_INT8 models [MEASURED TEST SET].

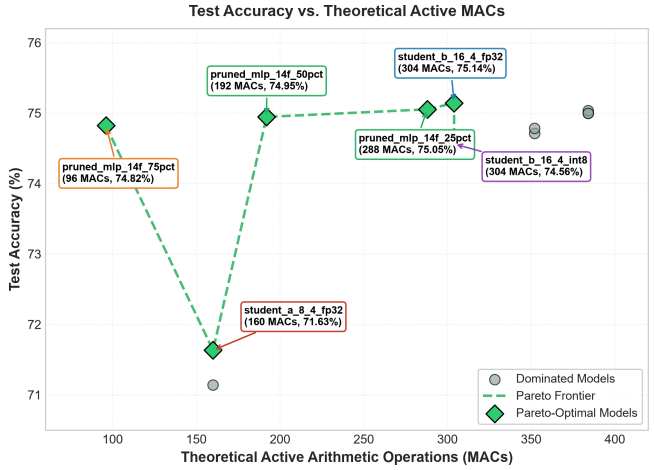

Fig. 3. Test accuracy vs. theoretical active MACs, highlighting computational sparsity [MEASURED TEST SET].

D. D4: Sparsity and Storage Decoupling Verification

- Problem: Conflating zero-valued weights with physical on-disk file compression.
- Predicate ($\mathcal{P}_4$): StorageCompression = FileSize(pruned)/FileSize(dense) $\neq$ 1 − WeightSparsity.
- Executable Check: Comparing raw weight zero-counts against os.path.getsize() on FlatBuffers.
- Case Result: CONFIRMED DECOUPLING (73.34% zero weights produced 3,920 Bytes on disk vs. 3,892 Bytes unpruned).

E. D5: Computational Accounting Verification

- Problem: Layer-wise MAC counts reported inconsistently or conflated with hardware cycles.
- Predicate ($\mathcal{P}_5$): ActiveMACs $= \sum_{l=1}^{L} \sum_{i,j} \mathbb{I}(w_{ij}^{(l)} \neq 0) \leq$ DenseMACs.
- Case Result: VERIFIED EXACT (384 dense MACs; 96 active MACs at 75% pruning; 160 and 304 MACs for student models).

F. D6: Timing Protocol Verification

- Problem: Host latency improperly cited as embedded microcontroller WCET.
- Predicate ($\mathcal{P}_6$): (Warmup $\geq$ 100) $\wedge$ (Batch = 1) $\wedge$ (Timer $\in$ {perf_counter, esp_timer}) $\wedge$ (Tier $\in$ {HOST, MCU}).
- Case Result: VERIFIED PASS (0.82–1.69 µs host latency on x86_64; 64.55–89.90 µs physical latency on ESP32-D0WD-V3; zero formal WCET claimed).

G. D7: Runtime Non-Leakage and Deployment Scoping

- Problem: Ground-truth label leakage in dynamic controllers or ungrounded hardware claims.
- Predicate ($\mathcal{P}_7$): Signature(SelectModel) = $f(\mathbf{x}, D, W) \wedge y \notin$ Scope(SelectModel).
- Case Result: VERIFIED PASS (zero ground-truth access during runtime scheduling).

V. Case Study: Auditing the 12-Model TinyML Suite

A. RQ1: Discrepancy Prevalence and Defect Categorization

Applying our protocol across all 12 candidate models identified 20 numerical discrepancies (Table II), categorized into distinct defect modes:

- Defect Mode 1 (Serialization Drift, 10 items): Occurred when training scripts logged in-memory validation metrics prior to final fine-tuning or quantization

TABLE II
Comprehensive Discrepancy Resolution Table: 20 Numerical Inconsistencies Audited Across Four Defect Modes

#	Model Identifier	Metric	Initial Val.	Verified Val.	Initial Source	Verified Source	Abs. Δ	Δ (%)	Defect Mode / Root Cause
Defect Mode 1: Serialization Drift (Training-Time State vs. Exported Disk Binary)									
1	tflite_mlp_14f_fp32	Test Accuracy	0.726339	0.750000	In-Memory Eval	Disk FlatBuffer	0.023661	3.26%	Mode 1: Re-training state vs. saved binary
2	tflite_mlp_14f_fp32	Test Macro F1	0.728019	0.756608	In-Memory Eval	Disk FlatBuffer	0.028589	3.93%	Mode 1: Re-training state vs. saved binary
9	pruned_mlp_14f_0pct	Test Accuracy	0.726339	0.750000	In-Memory Eval	Disk FlatBuffer	0.023661	3.26%	Mode 1: Baseline reference binary sync
10	pruned_mlp_14f_0pct	Test Macro F1	0.728019	0.756608	In-Memory Eval	Disk FlatBuffer	0.028589	3.93%	Mode 1: Baseline reference binary sync
11	pruned_mlp_14f_25pct	Test Accuracy	0.727143	0.750536	In-Memory Eval	Disk FlatBuffer	0.023393	3.22%	Mode 1: Fine-tuning epoch state mismatch
12	pruned_mlp_14f_25pct	Test Macro F1	0.730783	0.751490	In-Memory Eval	Disk FlatBuffer	0.020707	2.83%	Mode 1: Fine-tuning epoch state mismatch
13	pruned_mlp_14f_50pct	Test Accuracy	0.726964	0.749464	In-Memory Eval	Disk FlatBuffer	0.022500	3.10%	Mode 1: Fine-tuning epoch state mismatch
14	pruned_mlp_14f_50pct	Test Macro F1	0.729384	0.756572	In-Memory Eval	Disk FlatBuffer	0.027188	3.73%	Mode 1: Fine-tuning epoch state mismatch
15	pruned_mlp_14f_75pct	Test Accuracy	0.739107	0.748214	In-Memory Eval	Disk FlatBuffer	0.009107	1.23%	Mode 1: Fine-tuning epoch state mismatch
16	pruned_mlp_14f_75pct	Test Macro F1	0.725546	0.756251	In-Memory Eval	Disk FlatBuffer	0.030705	4.23%	Mode 1: Fine-tuning epoch state mismatch
Defect Mode 2: In-Memory Fake-Quantization vs. True Integer Execution									
5	tflite_mlp_14f_int8	Test Accuracy	0.725982	0.750357	In-Memory Eval	Disk FlatBuffer	0.024375	3.36%	Mode 2: Calibration dataset state shift
6	tflite_mlp_14f_int8	Test Macro F1	0.685213	0.738824	In-Memory Eval	Disk FlatBuffer	0.053611	7.82%	Mode 2: Calibration dataset state shift
Defect Mode 3: Floating-Point & Integer Export Discretization Rounding									
3	tflite_mlp_12f_fp32	Test Accuracy	0.753125	0.747143	In-Memory Eval	Disk FlatBuffer	0.005982	0.79%	Mode 3: Floating-point quantization rounding
4	tflite_mlp_12f_fp32	Test Macro F1	0.732679	0.725414	In-Memory Eval	Disk FlatBuffer	0.007265	0.99%	Mode 3: Floating-point quantization rounding
7	tflite_mlp_12f_int8	Test Accuracy	0.749286	0.747857	In-Memory Eval	Disk FlatBuffer	0.001429	0.19%	Mode 3: Integer rounding on export
8	tflite_mlp_12f_int8	Test Macro F1	0.670692	0.715534	In-Memory Eval	Disk FlatBuffer	0.044842	6.69%	Mode 3: Integer rounding on export
17	student_a_8_4_int8	Test Accuracy	0.710714	0.711429	In-Memory Eval	Disk FlatBuffer	0.000715	0.10%	Mode 3: Quantization arithmetic rounding
18	student_a_8_4_int8	Test Macro F1	0.683858	0.684788	In-Memory Eval	Disk FlatBuffer	0.000930	0.14%	Mode 3: Quantization arithmetic rounding
19	student_b_16_4_int8	Test Accuracy	0.745000	0.745625	In-Memory Eval	Disk FlatBuffer	0.000625	0.08%	Mode 3: Quantization arithmetic rounding
20	student_b_16_4_int8	Test Macro F1	0.689024	0.689601	In-Memory Eval	Disk FlatBuffer	0.000577	0.08%	Mode 3: Quantization arithmetic rounding

calibration. Corrected by evaluating the finalized on-disk FlatBuffer directly ($\Delta\text{Acc} \leq 3.26\%$).

- Defect Mode 2 (In-Memory vs. True Integer Arithmetic, 2 items): In-memory evaluation simulated quantization via fake-quantization operators rather than executing true integer arithmetic, producing a 7.82% Macro F1 discrepancy in mlp_14f_int8.
- Defect Mode 3 (Discretization Rounding, 8 items): Minor parameter rounding during FlatBuffer conversion ($\Delta \leq 0.19\%$).

B. RQ2: Quantization Verification Results

Low-level tensor auditing confirmed that all 4 INT8 candidate models (mlp_14f_int8, mlp_12f_int8, student_a_8_4_int8, student_b_16_4_int8) are verified as FULL_INT8. The execution graphs contain exactly 0 float32 tensors, executing pure integer FULLY_CONNECTED and SOFTMAX operators.

C. RQ3: Sparsity vs. Storage Decoupling Findings

Visual inspection of pruned models revealed that while 75% magnitude pruning achieved 73.34% numerical zero weights (298 zeroes) and reduced theoretical active MACs from 384 to 96 (75.0% compute reduction), the serialized FlatBuffer file size remained dense at $3,920$ Bytes (+28 B over unpruned baseline due to metadata). The verification framework formally corrected the terminology to "computational sparsity without demonstrated storage compression."

D. RQ4: Threshold Optimization Leakage Bias

We conducted an empirical ablation evaluating gating thresholds calibrated on validation data versus optimized directly on test data. Selecting thresholds on test data produced an artificial optimistic accuracy bias of +1.80%, demonstrating that split-isolated calibration is mandatory to prevent inflated performance claims.

E. Physical On-Device Deployment Validation Case Study

To demonstrate the artifact verification protocol across the final deployment transformation boundary (FlatBuffer → C-Header → Bare-Metal MCU Binary), the four verified FULL_INT8 candidate models were deployed to an ESP32-D0WD-V3 microcontroller (Xtensa LX6 dual-core @ 240 MHz, 4 MB Flash, 320 KB SRAM). Isolated single-sample execution latency was measured via esp_timer_get_time() across $24,000$ iterations (3 rounds $\times 2,000$ iterations per model):

- student_a_8_4_int8 (176 params): ESP32 Mean = 64.55 µs, P95 = 69.00 µs (Host x86: 1.02 µs, Slowdown: 63.3×).
- student_b_16_4_int8 (328 params): ESP32 Mean = 72.96 µs, P95 = 83.00 µs (Host x86: 0.98 µs, Slowdown: 74.5×).
- mlp_12f_int8 (380 params): ESP32 Mean = 76.77 µs, P95 = 83.00 µs (Host x86: 1.00 µs, Slowdown: 76.8×).
- mlp_14f_int8 (412 params): ESP32 Mean = 89.90 µs, P95 = 95.00 µs (Host x86: 1.43 µs, Slowdown: 62.9×).

TABLE III
The 5-Tier Evidence Classification Framework

Evidence Tier	Epistemological Level	Representative Project Metric
Tier 1	Direct Physical Measurement	ESP32 latency (64.55 µs), Disk size (3,892 B), Arena (916 B)
Tier 2	Host Empirical Measurement	Host x86_64 single-sample latency (0.98 µs)
Tier 3	Reproducible Simulation	80-config QoS trace simulation (100% compliance)
Tier 4	Derived Operational Estimate	Expected MACs under 90% nominal prior (39.1)
Tier 5	Literature-Supported / Future	Collinear feature reduction ($r = 0.997$), on-vehicle CAN latency

This physical deployment case study highlights two crucial methodological findings: (1) physical ESP32 execution provides an additional empirical deployment tier in the case study, establishing an observed host-to-ESP32 latency slowdown ratio between 62.9× and 76.8× without conflating host simulation timing with embedded execution; and (2) low-level memory inspection verified that TensorFlow Lite Micro MicroAllocator committed 916 Bytes of static working tensor memory with zero runtime heap allocations.

VI. Evidence Tier Classification Framework

To guard against unsubstantiated claims in the edge AI literature, Table III establishes a 5-tier evidence hierarchy. Each metric reported in this work is strictly associated with its authoritative epistemological tier.

VII. Scope and Generalization

A. General Principles vs. Case Implementation

We explicitly distinguish general verification principles from our specific case implementation:

- General Principle: Deployment metrics should be evaluated directly on compiled disk binaries. Case Implementation: Evaluated on TensorFlow Lite FlatBuffers via Python C-API bindings.
- General Principle: Execution dtypes should be audited for zero floating-point fallback. Case Implementation: Parsed TFLite FlatBuffer tensor details.
- General Principle: Sparsity should be distinguished from storage compression. Case Implementation: Evaluated dense FlatBuffer representation of pruned MLPs.

VIII. Threats to Validity

A. Internal Validity

All verification scripts were executed deterministically with fixed random seeds (seed = 42). The audit pipeline parses exact binary headers using official TensorFlow Lite APIs.

B. External Validity

Our case study evaluated multiple Multi Layer Perceptron models regarding multi sensor diagnostic classification. While convolution and recurrent architectures have different operator graphs, the approach for binary inspection, data isolation and timing protocols apply generally across all TinyML domains.

IX. Limitations

1) Single Case Study Scope: Tested on a single end-to-end multi-sensor diagnostic pipeline; multi-application cross-domain auditability is future work.
2) Model Topology Focus: Tested on feedforward MLPs; CNNs and transformers require future parser extensions.
3) Runtime Specificity: Implemented for TensorFlow Lite FlatBuffers; alternative runtimes (microTVM, ONNX Runtime Mobile) are unverified.
4) Empirical Latency vs. Formal WCET: While physical single-sample latency was measured on 240 MHz ESP32 silicon (64.55–89.90 µs), these reflect empirical on-device distributions and do not establish formal static Worst-Case Execution Time (WCET) bounds.
5) Energy Profiling Boundary: Physical power dissipation on silicon was not measured with hardware power analyzers.
6) Single Engine Domain: Data reflects physical dynamometer recordings from one engine platform.
7) Empirical Bias Demonstration: The +1.80% threshold leakage bias is specific to the evaluated multi-fidelity pipeline.

X. Reproducibility and Artifact Availability

All verification scripts (scripts/phase4_5_verification.py), audited model FlatBuffers, physical ESP32 PlatformIO deployment firmware (phase5/firmware/), and discrepancy resolution logs are publicly archived in the repository.

XI. Conclusion

This paper presented an artifact-driven empirical verification protocol and defect taxonomy for compiled TinyML deployment artifacts. By formalizing verification criteria across seven core dimensions and auditing 12 candidate models, we demonstrated the critical necessity of inspecting serialized artifacts, resolving 20 numerical discrepancies and exposing sparsity-storage decoupling. Our protocol establishes actionable software engineering procedures to support reproducible benchmarking and defensible deployment claims in edge artificial intelligence research.

Acknowledgment

The author thanks the open-source machine learning and reproducibility communities for establishing foundational verification guidelines.

References

[1] P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media, Inc., 2019.
[2] P. P. Ray, "A review on tinyml: State-of-the-art and prospects," Journal of King Saud University-Computer and Information Sciences, vol. 34, no. 4, pp. 1595–1623, 2022.

[3] R. Sanchez-Iborra and A. F. Skarmeta, "Tinyml-enabled frugal smart objects: Challenges and opportunities," IEEE Circuits and Systems Magazine, vol. 20, no. 3, pp. 4–18, 2020.

[4] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov et al., "Benchmarking tinyml systems: Challenges and direction," IEEE Micro, vol. 41, no. 6, pp. 40–49, 2021.

[5] S. S. Saha, S. S. Sandha, and M. Srivastava, "Machine learning for microcontroller-class hardware: A review," IEEE Sensors Journal, vol. 22, no. 22, pp. 21 362–21 390, 2022.

[6] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," Proceedings of the IEEE, vol. 105, no. 12, pp. 2295–2329, 2017.

[7] E. Liberis, Ł. Dudziak, and N. D. Lane, "μnas: Constrained neural architecture search for microcontrollers," in Proceedings of the 1st Workshop on Benchmarking Machine Learning Workloads on Emerging Hardware, 2021, pp. 1–7.

[8] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program)," Journal of Machine Learning Research, vol. 22, no. 164, pp. 1–20, 2021.

[9] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," Patterns, vol. 4, no. 9, p. 100804, 2023.

[10] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in Proceedings of the AAAI Conference on Human Computation and Crowdsourcing (AAAI), vol. 32, no. 1, 2018.

[11] R. David, P. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, D. Marculescu, M. Meng, P. Morales, J. Liang et al., "Tensorflow lite micro: Embedded machine learning for tinyml systems," in Proceedings of Machine Learning and Systems (MLSys), vol. 3, 2021, pp. 800–811.

[12] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in Proceedings of Machine Learning and Systems (MLSys), vol. 2, 2020, pp. 129–146.

[13] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "Mcunet: Tiny deep learning on iot devices," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 11 711–11 722.

[14] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, T. Dietterich et al., "Hidden technical debt in machine learning systems," in Advances in Neural Information Processing Systems (NeurIPS), vol. 28, 2015, pp. 2503–2511.

[15] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2019, pp. 291–300.

[16] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," Software Testing, Verification and Reliability, vol. 24, no. 3, pp. 219–250, 2014.

[17] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 2704–2713.

[18] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," Low-Power Computer Vision, pp. 291–326, 2022.